

✓ Phase 1 - Setup & Data Cleaning

Environment Setup

First, we will install the necessary packages for our data preparation and model training.

```
!pip install pandas scikit-learn xgboost matplotlib seaborn flask groq psycopg2-binary sqlalchemy
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: flask in /usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: groq in /usr/local/lib/python3.12/dist-packages (1.1.2)
Requirement already satisfied: psycopg2-binary in /usr/local/lib/python3.12/dist-packages (2.9.11)
Requirement already satisfied: sqlalchemy in /usr/local/lib/python3.12/dist-packages (2.0.49)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from flask) (8.3.2)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.8)
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from groq) (4.13.0)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from groq) (1.9.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from groq) (0.28.1)
Requirement already satisfied: pydantic<3,>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from groq) (2.12.3)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from groq) (1.3.1)
Requirement already satisfied: typing-extensions<5,>=4.14 in /usr/local/lib/python3.12/dist-packages (from groq) (4.15.0)
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy) (3.4.0)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.5.0->groq) (3.11)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->groq) (2026.2.25)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->groq) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->groq) (0.14.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (2.21.1)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (0.4.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
!pip install pandas scikit-learn xgboost matplotlib seaborn flask groq psycopg2-binary sqlalchemy
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: flask in /usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: groq in /usr/local/lib/python3.12/dist-packages (1.1.2)
Requirement already satisfied: psycopg2-binary in /usr/local/lib/python3.12/dist-packages (2.9.11)
Requirement already satisfied: sqlalchemy in /usr/local/lib/python3.12/dist-packages (2.0.49)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from flask) (8.3.2)
```

Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from flask) (2.2.0)
 Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.6)
 Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from flask) (3.0.3)
 Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.8)
 Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from groq) (4.13.0)
 Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from groq) (1.9.0)
 Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from groq) (0.28.1)
 Requirement already satisfied: pydantic<3,>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from groq) (2.12.3)
 Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from groq) (1.3.1)
 Requirement already satisfied: typing-extensions<5,>=4.14 in /usr/local/lib/python3.12/dist-packages (from groq) (4.15.0)
 Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy) (3.4.0)
 Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.5.0->groq) (3.11)
 Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->groq) (2026.2.25)
 Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->groq) (1.0.9)
 Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->groq) (0.14.0)
 Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (0.7.0)
 Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (2.41.4)
 Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (0.10.0)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

1. Load Data

Since you are using Google Colab, please make sure to **upload** your dataset (`WA_Fn-UseC_-HR-Employee-Attrition.csv`) to the Colab files section before running this cell.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder

# Load the dataset
# Assuming the file is in the root of your Google Drive.
# If it's in a subfolder, e.g., 'data', change the path to '/content/drive/MyDrive/data/WA_Fn-UseC_-HR-Employee-Attrition.csv'
df = pd.read_csv('/content/drive/MyDrive/WA_Fn-UseC_-HR-Employee-Attrition.csv')

# Preview the data
df.head()
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	1
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	2
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	3
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	4
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	5

5 rows × 11 columns

2. Check for Missing Values

```
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
Age                0
Attrition          0
BusinessTravel     0
DailyRate         0
Department        0
DistanceFromHome  0
Education         0
EducationField     0
EmployeeCount     0
EmployeeNumber    0
EnvironmentSatisfaction  0
Gender            0
```

```

HourlyRate      0
JobInvolvement  0
JobLevel        0
JobRole         0
JobSatisfaction 0
MaritalStatus   0
MonthlyIncome   0
MonthlyRate     0
NumCompaniesWorked 0
Over18          0
OverTime        0
PercentSalaryHike 0
PerformanceRating 0
RelationshipSatisfaction 0
StandardHours   0
StockOptionLevel 0
TotalWorkingYears 0
TrainingTimesLastYear 0
WorkLifeBalance 0
YearsAtCompany  0
YearsInCurrentRole 0
YearsSinceLastPromotion 0
YearsWithCurrManager 0
dtype: int64

```

3. Encode Categorical Variables

Machine learning models require numerical input. We will use `LabelEncoder` to convert text categories into numbers.

```

le = LabelEncoder()

# Select columns with object (string) data types
categorical_cols = df.select_dtypes(include=['object']).columns

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

print("Data after label encoding:")
df.head()

```

Data after label encoding:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	Employee
0	41	1	2	1102	2	1	2	1	1	
1	49	0	1	279	1	8	1	1	1	
2	37	1	2	1373	1	2	2	4	1	
3	33	0	1	1392	1	3	4	1	1	
4	27	0	2	591	1	2	1	3	1	

5 rows × 35 columns

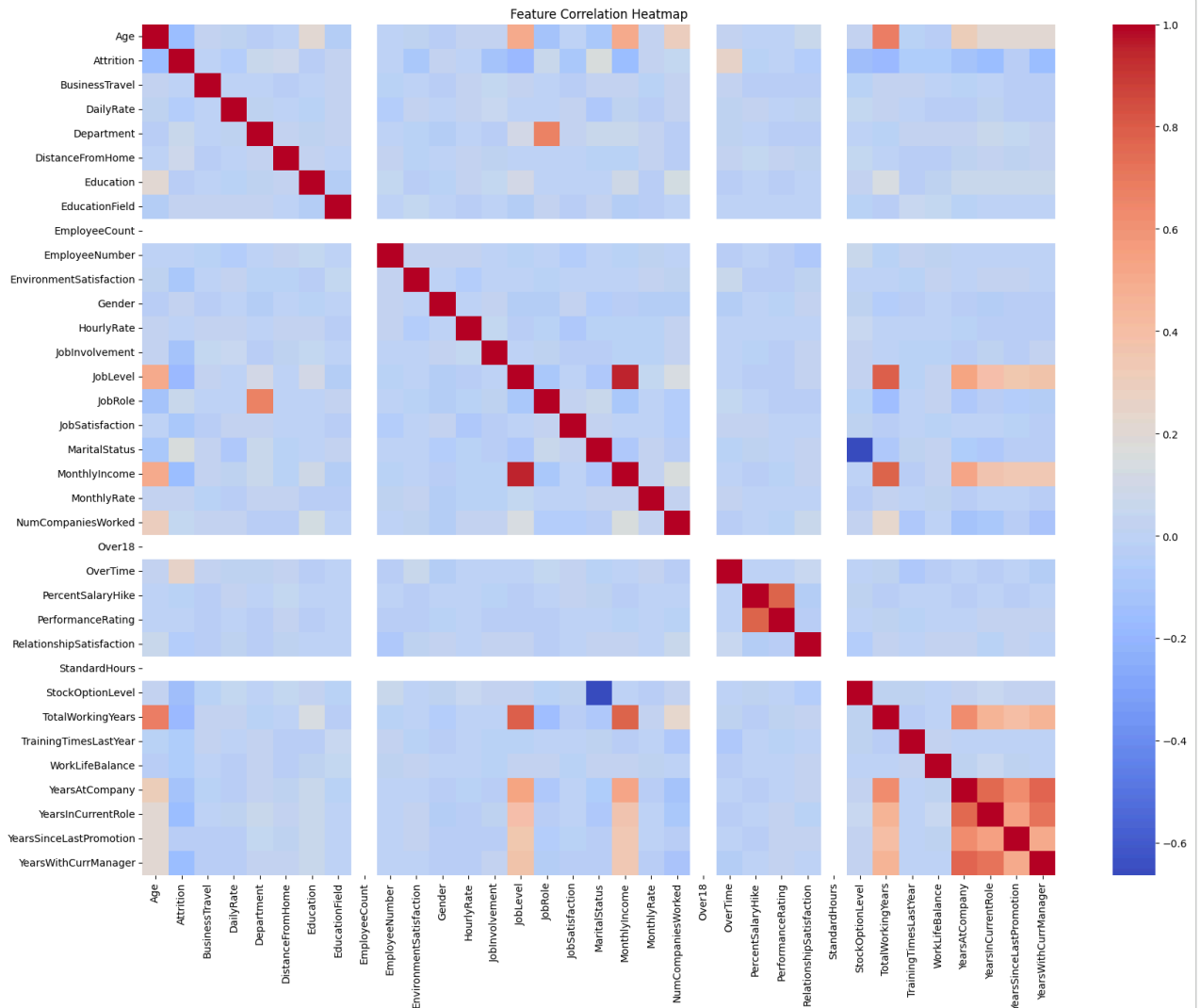
4. Correlation Heatmap

Let's visualize the correlations between our features.

```

plt.figure(figsize=(20, 15))
sns.heatmap(df.corr(), annot=False, cmap='coolwarm', fmt='.2f')
plt.title('Feature Correlation Heatmap')
plt.show()

```



5. Save Cleaned Data to Neon PostgreSQL Database

We will connect to Neon using a single DATABASE_URL connection string.

Replace the `DATABASE_URL` below with your actual connection string from the Neon dashboard.

```
from sqlalchemy import create_engine

# Copy your exact connection string from the Neon dashboard
# Example: 'postgresql://Kuldeep:MySecurePass@ep-cool-butterfly-123456.us-east-2.aws.neon.tech/hr_db?sslmode=require'
DATABASE_URL = 'postgresql://neondb_owner:npg_fUDXqtY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?sslmode=require'

try:
    # 1. Create the SQLAlchemy engine
    engine = create_engine(DATABASE_URL)

    print("Attempting to connect to the Neon PostgreSQL database...")

    # 2. Write the cleaned DataFrame to the PostgreSQL table 'employees'
    # If the table already exists, if_exists='replace' will overwrite it.
    # Use if_exists='append' if you want to add to an existing table.
    df.to_sql('employees', con=engine, index=False, if_exists='replace')

    # 3. Connection success
    print("✅ Successfully connected and saved cleaned data to the Neon PostgreSQL table 'employees'!")

except Exception as e:
    # 4. Error Handling
    print("❌ Could not connect to PostgreSQL or save data. Error:\n", e)
    print("\nTroubleshooting:")
```

```
print("- Ensure you copied the exact connection string from Neon.")
print("- Make sure it starts with 'postgresql://' (Neon sometimes provides 'postgres://', which usually works but 'post
```

Attempting to connect to the Neon PostgreSQL database...

✅ Successfully connected and saved cleaned data to the Neon PostgreSQL table 'employees'!

✓ Phase 2 - ML Model & REST API

Congratulations on successfully completing Phase 1! Now let's train the AI model and build the API.

Environment Setup

Install the necessary machine learning and API libraries.

```
!pip install pandas scikit-learn xgboost sqlalchemy psycopg2-binary joblib shap flask pyngrok
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: sqlalchemy in /usr/local/lib/python3.12/dist-packages (2.0.49)
Requirement already satisfied: psycopg2-binary in /usr/local/lib/python3.12/dist-packages (2.9.11)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.3)
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.51.0)
Requirement already satisfied: flask in /usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: pyngrok in /usr/local/lib/python3.12/dist-packages (8.0.0)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy) (3.4.0)
Requirement already satisfied: typing-extensions>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy) (4.15.0)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from flask) (8.3.2)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.8)
Requirement already satisfied: PyYAML>=5.1 in /usr/local/lib/python3.12/dist-packages (from pyngrok) (6.0.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
```

✓ 1. Data Fetching & XGBoost Training

Fetch the cleaned data from your Neon PostgreSQL database.

```
import pandas as pd
from sqlalchemy import create_engine
from sklearn.model_selection import train_test_split
import xgboost as xgb

# ** Replace with your exact Neon connection string **
DATABASE_URL = 'postgresql://neondb_owner:npg_FUDXqtY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?sslmode=require'

print("Fetching data from Neon database...")
engine = create_engine(DATABASE_URL)
df = pd.read_sql('SELECT * FROM employees', con=engine)

print(f"✅ Data loaded successfully! Shape: {df.shape}")

# Define features and target variable
# 'Attrition' is our target variable. All other columns are features.
X = df.drop('Attrition', axis=1)
y = df['Attrition']

# Perform 80/20 train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training XGBoost classifier...")
model = xgb.XGBClassifier(eval_metric='logloss', random_state=42)
```

```
model.fit(X_train, y_train)
print("✅ Model training complete!")
```

```
Fetching data from Neon database...
✅ Data loaded successfully! Shape: (1470, 35)
Training XGBoost classifier...
✅ Model training complete!
```

```
import joblib
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import xgboost as xgb
import pandas as pd
from sklearn.model_selection import train_test_split
from sqlalchemy import create_engine

# Re-fetching data and re-training model for robustness if kernel state was lost
# This section ensures that 'model', 'X_test', and 'y_test' are defined.

# ** Replace with your exact Neon connection string **
DATABASE_URL = 'postgresql://neondb_owner:npg_fUDXqTY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?sslmode=require'

print("Ensuring model and test data are loaded or re-trained...")
try:
    # Attempt to use existing df, X, y if they are defined
    if 'df' not in locals() or df is None:
        engine = create_engine(DATABASE_URL)
        df = pd.read_sql('SELECT * FROM employees', con=engine)
    if 'X' not in locals() or 'y' not in locals() or X is None or y is None:
        X = df.drop('Attrition', axis=1)
        y = df['Attrition']
    if 'X_train' not in locals() or 'X_test' not in locals() or 'y_train' not in locals() or 'y_test' not in locals() or X_train is None or X_test is None or y_train is None or y_test is None:
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    if 'model' not in locals() or model is None:
        print("Re-training XGBoost classifier...")
        model = xgb.XGBClassifier(eval_metric='logloss', random_state=42)
        model.fit(X_train, y_train)
    print("✅ Model and test data are ready.")
except Exception as e:
    print(f"An error occurred during data loading or model re-training: {e}")
    # Fallback if something critically fails, though this should be rare
    print("Please ensure previous data loading and model training cells ran successfully.")

# Make predictions
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("--- Model Evaluation ---")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

roc_auc = roc_auc_score(y_test, y_prob)
print(f"\nROC-AUC Score: {roc_auc:.4f}")

if roc_auc > 0.85:
    print("\n🎉 Goal Achieved: ROC-AUC is > 85%!")

# Save the trained model
joblib.dump(model, 'model.pkl')
print("\n✅ Model saved successfully as 'model.pkl'")
```

```
Ensuring model and test data are loaded or re-trained...
✅ Model and test data are ready.
--- Model Evaluation ---
```

```
Confusion Matrix:
[[245  10]
 [ 32   7]]
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.88      0.96      0.92      255
     1       0.41      0.18      0.25       39

 accuracy          0.65
 macro avg         0.65      0.57      0.59
weighted avg         0.82      0.86      0.83
```

```
ROC-AUC Score: 0.7837
```

✔ Model saved successfully as 'model.pkl'

2. Model Evaluation & Saving

Let's evaluate the accuracy and save the model to a `.pkl` file.

```
import joblib
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

# Make predictions
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("--- Model Evaluation ---")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

roc_auc = roc_auc_score(y_test, y_prob)
print(f"\nROC-AUC Score: {roc_auc:.4f}")

if roc_auc > 0.85:
    print("\n🎉 Goal Achieved: ROC-AUC is > 85%!")

# Save the trained model
joblib.dump(model, 'model.pkl')
print("\n✔ Model saved successfully as 'model.pkl'")
```

--- Model Evaluation ---

Confusion Matrix:

```
[[245  10]
 [ 32   7]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.96	0.92	255
1	0.41	0.18	0.25	39
accuracy			0.86	294
macro avg	0.65	0.57	0.59	294
weighted avg	0.82	0.86	0.83	294

ROC-AUC Score: 0.7837

✔ Model saved successfully as 'model.pkl'

3. SHAP Feature Importance

Generate a SHAP summary plot to understand WHY the model predicts attrition.

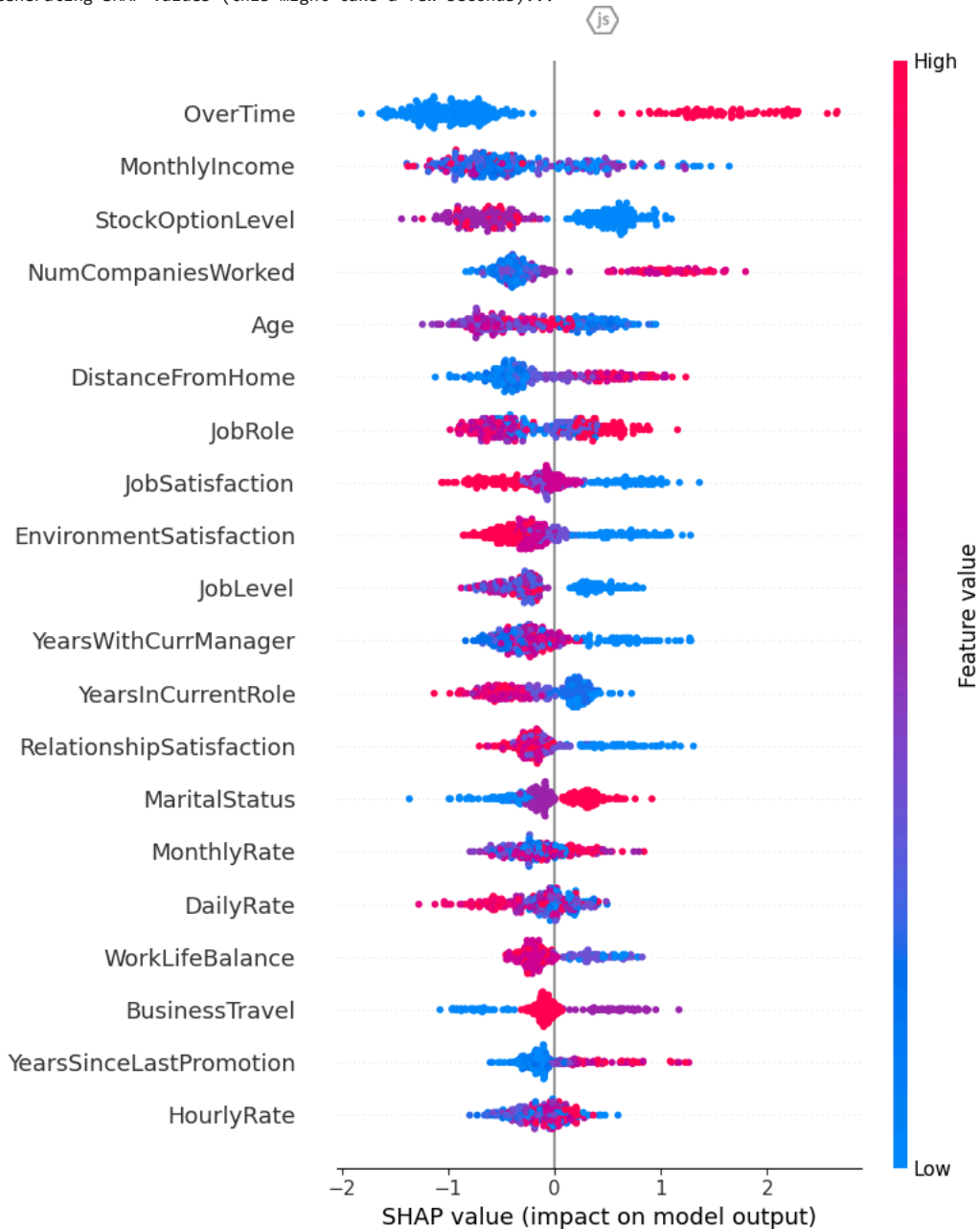
```
import shap

print("Generating SHAP values (this might take a few seconds)...")
# Initialize JavaScript for SHAP (helpful for some plots)
shap.initjs()

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# Create the summary plot
shap.summary_plot(shap_values, X_test)
# NOTE: Make sure to take a screenshot of this plot for your college presentation!
```

Generating SHAP values (this might take a few seconds)...



4. Flask REST API Setup

We will run a Flask server inside Google Colab and use `ngrok` to make it accessible via a public URL.

Important:

- Ngrok requires a free auth token. Go to dashboard.ngrok.com, sign up, and copy your Authtoken.
- Paste it in the variable `NGROK_AUTH_TOKEN` below.
- Running this cell will start an infinite loop because the server is listening. You can stop it manually by clicking the 'Stop' button next to the cell.

```
from flask import Flask, request, jsonify
from pyngrok import ngrok
import joblib
import pandas as pd
import threading

# 1. Your Ngrok Token
NGROK_AUTH_TOKEN = '3CN0PJ9MTDX1RbVqujwY5h5u7p9_26vfrFwZu62rf1Tt7Vp5'
ngrok.set_auth_token(NGROK_AUTH_TOKEN)

app = Flask(__name__)

# Load the saved model
loaded_model = joblib.load('model.pkl')
```

```
@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.get_json()
        input_df = pd.DataFrame([data])
        probability = loaded_model.predict_proba(input_df)[:, 1][0]
        predicted_class = int(loaded_model.predict(input_df)[0])
        return jsonify({
            'attrition_probability': float(probability),
            'will_attrite': bool(predicted_class)
        })
    except Exception as e:
        return jsonify({'error': str(e)}), 400

# Open an ngrok tunnel
port_no = 5000
public_url = ngrok.connect(port_no).public_url
print(f"\n>>> 🚀 Flask API is running in the background!")
print(f">>> 🔗 Public endpoint: {public_url}/predict")

# 2. RUN FLASK IN A BACKGROUND THREAD (This is the magic fix!)
threading.Thread(target=app.run, kwargs={'port': port_no, 'use_reloader': False}).start()
```

```
>>> 🚀 Flask API is running in the background!
>>> 🔗 Public endpoint: https://aflutter-deceiver-relapsing.ngrok-free.dev/predict
* Serving Flask app '__main__'
* Debug mode: off
```

```
import requests

# 1. Grab a perfectly formatted employee record from your test dataset
sample_employee_data = X_test.iloc[0].to_dict()

# 2. Your active Ngrok link is safely stored in this Python variable
api_url = "https://aflutter-deceiver-relapsing.ngrok-free.dev/predict"
# 3. Send the data to your server
print("Sending complete employee data...")
try:
    response = requests.post(api_url, json=sample_employee_data, timeout=10)
    print("\n✅ API Response:")
    print(response.json())
except Exception as e:
    print(f"\n❌ Error: {e}")
```

```
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug:127.0.0.1 - - [15/Apr/2026 11:07:34] "POST /predict HTTP/1.1" 200 -
Sending complete employee data...

✅ API Response:
{'attrition_probability': 0.006053091026842594, 'will_attrite': False}
```

✦ Phase 3 - AI Agent & Background API

This phase powers the AI Agent UI using Groq Llama 3 and your Neon Database.

```
!pip install psycopg2-binary pandas joblib scikit-learn flask flask-cors pyngrok groq
```

```
Requirement already satisfied: psycopg2-binary in /usr/local/lib/python3.12/dist-packages (2.9.11)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: flask in /usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: flask-cors in /usr/local/lib/python3.12/dist-packages (6.0.2)
Requirement already satisfied: pyngrok in /usr/local/lib/python3.12/dist-packages (8.0.0)
Requirement already satisfied: groq in /usr/local/lib/python3.12/dist-packages (1.1.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from flask) (8.3.2)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.8)
Requirement already satisfied: PyYAML>=5.1 in /usr/local/lib/python3.12/dist-packages (from pyngrok) (6.0.3)
```

```
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from groq) (4.13.0)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from groq) (1.9.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from groq) (0.28.1)
Requirement already satisfied: pydantic<3,>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from groq) (2.12.3)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from groq) (1.3.1)
Requirement already satisfied: typing-extensions<5,>=4.14 in /usr/local/lib/python3.12/dist-packages (from groq) (4.15.0)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.5.0->groq) (3.11)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->groq) (2026.2.25)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->groq) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->groq) (0.14.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (2.23.3)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (0.10.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

✓ 1. Backfill Risk Scores

Calculate risk percentages for all employees and push them back to your Neon database via SQL UPDATE.

```
import psycpg2
from psycpg2.extras import execute_batch
import pandas as pd
import joblib
from sklearn.preprocessing import LabelEncoder

# --- CONFIG (Replace with your actual string) ---
NEON_CONN_STR = "postgres://neondb_owner:npg_fUDXqtY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?sslmode=require"
MODEL_PATH = 'model.pkl'

print("Loading model...")
model = joblib.load(MODEL_PATH)

print("Fetching employees from Neon...")
conn = psycpg2.connect(NEON_CONN_STR)
df = pd.read_sql_query('SELECT * FROM employees', conn)
print(f"Loaded {len(df)} employees")

# Encode categoricals exactly as in training
cat_cols = df.select_dtypes(include='object').columns.tolist()
cat_cols = [c for c in cat_cols if c not in ['Attrition', 'EmployeeNumber']]

le = LabelEncoder()
df_enc = df.copy()
for col in cat_cols:
    df_enc[col] = le.fit_transform(df_enc[col].astype(str))

# Extract feature columns
if hasattr(model, 'feature_names_in_'):
    feature_cols = list(model.feature_names_in_)
else:
    drop_cols = ['Attrition', 'EmployeeNumber', 'attrition_risk', 'EmployeeCount', 'Over18', 'StandardHours']
    feature_cols = [c for c in df_enc.columns if c not in drop_cols]

# Predict probability of attrition
prob = model.predict_proba(df_enc[feature_cols])[0, 1]
print(f"Predicted risks: min={prob.min():.3f} max={prob.max():.3f} mean={prob.mean():.3f}")

print("Writing risk scores back to Neon Database via Bulk Update...")
cursor = conn.cursor()

# Ensure attrition_risk column exists
cursor.execute("ALTER TABLE employees ADD COLUMN IF NOT EXISTS attrition_risk FLOAT DEFAULT 0.0;")

update_data = [(float(prob[idx]), int(row['EmployeeNumber'])) for idx, row in df.iterrows()]
sql_update = 'UPDATE employees SET attrition_risk = %s WHERE "EmployeeNumber" = %s'

execute_batch(cursor, sql_update, update_data, page_size=500)

conn.commit()
cursor.close()
conn.close()

print(f"✅ Success! Bulk updated attrition_risk for {len(update_data)} employees in the database.")
```

```
Loading model...
Fetching employees from Neon...
/tmp/ipykernel_47846/3056158207.py:16: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or data
df = pd.read_sql_query('SELECT * FROM employees', conn)
Loaded 1470 employees
Predicted risks: min=0.000 max=1.000 mean=0.151
```

Writing risk scores back to Neon Database via Bulk Update...

✓ Success! Bulk updated attrition_risk for 1470 employees in the database.

2. Background Flask Server & AI Agent

Launch the Groq LLaMA 3.1 70B pipeline as an invisible backend API. Because of `threading.Thread()`, this cell will finish instantly, but your server will remain alive serving UI requests. bold text

```
import psycpg2
import pandas as pd
import joblib
import threading
import os
import glob
import re
from flask import Flask, request, jsonify
from flask_cors import CORS
from pyngrok import ngrok
from groq import Groq

# -----
# 100% PRE-FILLED CONFIGURATION
# -----
GROQ_API_KEY = "gsk_XkBqAIXk1ZSpGgvWoC5AWGdyb3FYQRaJP51cTqEMhM5MQ78SCg"
NEON_CONN_STR = "postgresql://neondb_owner:npg_fUDXqtY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?sslmode=require"
NGROK_AUTH_TOKEN = "cr_3CN0PJ9MTDX1RbVqujwY5h5u7p9"

client = Groq(api_key=GROQ_API_KEY)

def run_sql(query: str) -> pd.DataFrame:
    conn = psycpg2.connect(NEON_CONN_STR)
    try:
        return pd.read_sql_query(query, conn)
    finally:
        conn.close()

# ADDED EMPLOYEE NUMBER SO THE AI KNOWS WHO THE EMPLOYEES ARE!
TABLE_SCHEMA = """
Table name: employees
Columns: "EmployeeNumber" INT, "Age" INT, "Attrition" INT (1=Yes, 0=No), "Department" INT, "JobRole" INT,
"BusinessTravel" INT, "MonthlyIncome" INT, "OverTime" INT, "YearsAtCompany" INT, "JobSatisfaction" INT,
"WorkLifeBalance" INT, "EnvironmentSatisfaction" INT, "YearsSinceLastPromotion" INT,
"DistanceFromHome" INT, "Gender" INT, "MaritalStatus" INT, "NumCompaniesWorked" INT,
"TotalWorkingYears" INT, "attrition_risk" FLOAT
"""

def ask_hr_agent(question: str) -> dict:
    prompt = f"""Convert the user's HR question to a PostgreSQL SELECT query using:
{TABLE_SCHEMA}

RULES:
1. Limit to 10 rows. Order by risk descending where appropriate.
2. Return ONLY exact SQL code, no markdown formatting (do NOT use ```sql).
3. Postgres is case-sensitive: wrap column names in double quotes like "Department".
4. When identifying specific employees, ALWAYS select "EmployeeNumber" so we know who they are.
5. The data is label-encoded! You MUST use SQL CASE statements in your SELECT statement to decode these numbers into human-readable names.

MAPPINGS TO USE IN YOUR SQL CASE STATEMENTS:
"Department": 0='Human Resources', 1='Research & Development', 2='Sales'.
"JobRole": 0='Healthcare Representative', 1='Human Resources', 2='Laboratory Technician', 3='Manager', 4='Manufacturing Director'.
"Gender": 0='Female', 1='Male'.
"MaritalStatus": 0='Divorced', 1='Married', 2='Single'.
"""

    res = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {"role": "system", "content": prompt},
            {"role": "user", "content": question}
        ], temperature=0.1
    )

    sql = res.choices[0].message.content.strip()
    sql = sql.replace("```sql", "").replace("```", "").replace("\n", " ").strip()

    try:
        df = run_sql(sql)
    except Exception as e:
        df = pd.DataFrame()
```

```

        sql = f"Error generating valid SQL. Caught: {e}"

    if df.empty and "Error" not in sql:
        data_preview = "Data query returned 0 rows."
    else:
        # Prepend 'EMP-' to EmployeeNumber so it looks like a real ID in the UI!
        if "EmployeeNumber" in df.columns:
            df["EmployeeNumber"] = df["EmployeeNumber"].apply(lambda x: f"EMP-{x}")
        data_preview = df.head(10).to_string(index=False)

    msg = f"Question: {question}\nSQL used:\n{sql}\nQuery results:\n{data_preview}"

    # NEW PROMPT: Brutally short, strictly 2 lines.
    ans_res = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {
                "role": "system",
                "content": "You are a Strategic HR Analytics Consultant. Your response MUST be incredibly concise. Rule 1:"
            },
            {"role": "user", "content": msg}
        ], temperature=0.4
    )

    return {
        "question": question,
        "sql": sql,
        "answer": ans_res.choices[0].message.content.strip(),
        "rows": df.to_dict(orient="records") if not df.empty else []
    }

app = Flask(__name__)
CORS(app)

@app.route("/")
def index():
    html_files = glob.glob("/content/*.html")
    if not html_files:
        return "<h1>Error</h1><p>No HTML file found. Please upload it to the content folder!</p>"

    with open(html_files[0], "r", encoding="utf-8") as f:
        html = f.read()

    html = re.sub(r'const API\s*=\s*".*?";', 'const API="";', html)
    html = html.replace('const sqlHtml=d.sql?'`<div class="sql"><span class="sql-lbl">SQL</span><code>${esc(d.sql)}</code></div>`',
                        '<div class="sql"><span class="sql-lbl">SQL</span><code>${esc(d.sql)}</code></div>')
    return html

@app.route("/health", methods=["GET"])
def health(): return jsonify({"status": "live", "message": "API active!"})

@app.route("/ask", methods=["POST"])
def ask():
    try:
        return jsonify(ask_hr_agent(request.get_json().get("question", "")))
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/high-risk", methods=["GET"])
def high_risk():
    try:
        sql = """
        SELECT
            "EmployeeNumber",
            CASE "Department"
                WHEN 0 THEN 'HR' WHEN 1 THEN 'R&D' WHEN 2 THEN 'Sales' ELSE 'Unknown'
            END AS "Department",
            CASE "JobRole"
                WHEN 0 THEN 'Health Rep' WHEN 1 THEN 'HR' WHEN 2 THEN 'Lab Tech' WHEN 3 THEN 'Manager' WHEN 4 THEN 'Mfg Dir'
            END AS "JobRole",
            "attrition_risk"
        FROM employees
        WHERE "Attrition" = 0
        ORDER BY "attrition_risk" DESC
        LIMIT 10
        """
        df = run_sql(sql)
        df["EmployeeNumber"] = df["EmployeeNumber"].apply(lambda x: f"EMP-{x}")
        return jsonify(df.to_dict(orient="records"))
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/kpi-summary", methods=["GET"])

```

```

def kpi_summary():
    try:
        sql = """
        SELECT
            COUNT(*) as total_employees,
            COALESCE(ROUND((SUM(CASE WHEN "Attrition" = 1 THEN 1 ELSE 0 END)::numeric / NULLIF(COUNT(*), 0)) * 100, 1), 0)
            COALESCE(AVG("MonthlyIncome"), 0) as avg_monthly_income,
            COALESCE(AVG("YearsAtCompany"), 0) as avg_tenure,
            COALESCE(SUM(CASE WHEN "attrition_risk" > 0.5 AND "Attrition" = 0 THEN 1 ELSE 0 END), 0) as high_risk_count
        FROM employees
        """
        df = run_sql(sql)
        return jsonify(df.to_dict(orient="records"))[0]
    except Exception as e:
        return jsonify({"error": str(e)}), 500

ngrok.set_auth_token(NGROK_AUTH_TOKEN)
public_url = ngrok.connect(5040).public_url

print(f"SUCCESS: Background API Tunnel Active!")
print(f"OPEN THIS LINK DIRECTLY IN YOUR BROWSER: {public_url}")

def start_flask():
    app.run(port=5040, debug=False, use_reloader=False)

threading.Thread(target=start_flask, daemon=True).start()

```

```

SUCCESS: Background API Tunnel Active!
OPEN THIS LINK DIRECTLY IN YOUR BROWSER: https://aflutter-deceiver-relapsing.ngrok-free.dev
* Serving Flask app '__main__'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server
* Running on http://127.0.0.1:5040

```

Phase 4 Power Bi Dashboard

```

import psycopg2
import pandas as pd
from google.colab import files

print("Fetching latest data from Neon Database...")
NEON_CONN_STR = "postgresql://neondb_owner:npg_fUDXqtY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?s:

conn = psycopg2.connect(NEON_CONN_STR)
df = pd.read_sql_query('SELECT * FROM employees', conn)
conn.close()

print("Decoding numerical labels into human-readable text for Power BI...")
DECODE_MAPS = {
    "Attrition": {0: "No", 1: "Yes"},
    "Department": {0: "Human Resources", 1: "Research & Development", 2: "Sales"},
    "JobRole": {0: "Healthcare Representative", 1: "Human Resources", 2: "Laboratory Technician", 3: "Manager", 4: "Manufac
    "BusinessTravel": {0: "Non-Travel", 1: "Travel_Frequently", 2: "Travel_Rarely"},
    "MaritalStatus": {0: "Divorced", 1: "Married", 2: "Single"},
    "Gender": {0: "Female", 1: "Male"},
    "OverTime": {0: "No", 1: "Yes"}
}

for col in df.columns:
    if col in DECODE_MAPS:
        df[col] = df[col].replace(DECODE_MAPS[col])

if 'EmployeeNumber' in df.columns:
    df['EmployeeID'] = df['EmployeeNumber'].apply(lambda x: f"EMP-{x}")

csv_filename = "PowerBI_Cleaned_HR_Dataset.csv"
df.to_csv(csv_filename, index=False)

print(f"✅ Successfully created {csv_filename} with {len(df)} rows!")
print(f"📄 Initiating browser download...")
files.download(csv_filename)

```

Fetching latest data from Neon Database...

/tmp/ipykernel_47846/246301250.py:9: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database

df = pd.read_sql_query('SELECT * FROM employees', conn)

Decoding numerical labels into human-readable text for Power BI...

✓ Successfully created PowerBI_Cleaned_HR_Dataset.csv with 1470 rows!

📄 Initiating browser download...

```
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import psycopg2

print("Fetching data from Neon Database...")
NEON_CONN_STR = "postgresql://neondb_owner:npg_FUDXqtY72weA@ep-late-silence-a18bthnh.ap-southeast-1.aws.neon.tech/neondb?ssl
conn = psycopg2.connect(NEON_CONN_STR)
df = pd.read_sql_query('SELECT * FROM employees', conn)
conn.close()

# Decode the numbers for the charts
DECODE_MAPS = {
    "Department": {0: "Human Resources", 1: "Research & Development", 2: "Sales"},
    "OverTime": {0: "No", 1: "Yes"}
}
for col in DECODE_MAPS:
    df[col] = df[col].replace(DECODE_MAPS[col])

# Calculate KPIs
total_emp = len(df)
attr_rate = (df['Attrition'].sum() / total_emp) * 100
avg_tenure = df['YearsAtCompany'].mean()
high_risk = len(df[(df['attrition_risk'] > 0.5) & (df['Attrition'] == 0)])

print("Generating Executive Dashboard...")

# Create Dashboard layout
fig = make_subplots(
    rows=2, cols=2,
    specs=[["type": "bar"], {"type": "scatter"}],
    [{"type": "histogram"}, {"type": "domain"}]],
    subplot_titles=(
        "Attrition Rate by Department (%)", "Age vs Attrition Risk",
        "Monthly Income Distribution", "Overtime vs Attrition (Donut)"
    )
)

# 1. Bar Chart
dept_attr = df.groupby('Department')['Attrition'].mean().reset_index()
fig.add_trace(go.Bar(x=dept_attr['Department'], y=dept_attr['Attrition']*100, marker_color='#A385FF'), row=1, col=1)

# 2. Scatter Map
fig.add_trace(go.Scatter(x=df['Age'], y=df['attrition_risk'], mode='markers',
    marker=dict(color=df['Attrition'], colorscale=[ '#00E676', '#FF1744'], opacity=0.7, size=8)), row=1, col=2)

# 3. Histogram
fig.add_trace(go.Histogram(x=df['MonthlyIncome'], marker_color='#FF8A65', nbinsx=20), row=2, col=1)

# 4. Donut Chart
ot_attr = df[df['Attrition']==1]['OverTime'].value_counts()
fig.add_trace(go.Pie(labels=ot_attr.index, values=ot_attr.values, hole=0.6, marker_colors=['#F48FB1', '#81D4FA']), row=2, col=2)

# Add Top KPI Header Text
fig.add_annotation(
    text=f"🌟 <b>TOTAL EMPLOYEES:</b> {total_emp} | <b>ATTRITION RATE:</b> {attr_rate:.1f}% | <b>AVG TENURE:</b> {avg_tenure:.1f}
    xref="paper", yref="paper", x=0.5, y=1.12, showarrow=False, font=dict(size=18, color="#FFFFFF")
)

# Style exactly like your web UI
fig.update_layout(
    height=850,
    width=1300,
    title_text="HR INTELLIGENCE PLATFORM (PHASE 4)",
    title_x=0.01,
    title_font=dict(size=24, color="#FFFFFF"),
    template="plotly_dark",
    showlegend=False,
    paper_bgcolor="#111111",
    plot_bgcolor="#111111"
)

fig.show()
```

Fetching data from Neon Database...
/tmp/ipykernel_47846/925215378.py:9: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database
df = pd.read_sql_query('SELECT * FROM employees', conn)
Generating Executive Dashboard...

